ОТЧЁТ ПО АВТОМАТИЗИРОВАННОМУ ТЕСТИРОВАНИЮ

Веб-платформа Polymarket

1. Описание предмета тестирования

Объектом тестирования является веб-платформа для торговли предсказаниями Polymarket (URL: https://polymarket.com/). Назначение сайта – предоставление возможности делать ставки на исход реальных событий с использованием блокчейн-технологий.

Основные функции для пользователя:

• Просмотр рынков: отображение активных рынков предсказаний, сгруппированных по категориям

• Поиск событий: поиск интересующего рынка по ключевым словам

• Просмотр деталей рынка: описание события, вероятности исходов, объём торгов

• Торговля: покупка и продажа долей (Yes/No) на исход события

2. Описание окружения тестирования

Параметр	Значение
Операционная система	Linux (Ubuntu/Debian)
Браузер	Chrome 146.0.7680.153 (Chrome for Testing)
Python	3.13.5
Selenium	4.41.0
Pytest	9.0.2
Дата тестирования	2026-03-23 22:29

3. Сценарии использования

Сценарий 1: Просмотр рынков по категории (позитивный)

№	Действие	Ожидаемый результат
1	Открыть главную страницу Polymarket	Загружается главная страница
2	Нажать на фильтр категории "Politics"	Список рынков фильтруется
3	Выбрать карточку рынка	Открывается детальная страница рынка

Сценарий 2: Поиск рынка предсказаний (позитивный)

№	Действие	Ожидаемый результат
1	Открыть главную страницу	Загружается главная страница
2	Нажать на иконку поиска	Появляется поле ввода
3	Ввести "Bitcoin" и нажать Enter	Отображаются результаты поиска
4	Выбрать рынок из списка	Открывается страница рынка

Сценарий 3: Просмотр деталей рынка и истории цен (позитивный)

№	Действие	Ожидаемый результат
1	Открыть главную страницу	Загружается главная страница
2	Нажать на карточку рынка	Открывается страница рынка
3	Просмотреть вероятности исходов	Отображаются Yes/No вероятности
4	Переключить период на графике (1M)	График обновляется

Сценарий 4: Попытка торговли без авторизации (негативный)

№	Действие	Ожидаемый результат
1	Открыть страницу рынка	Страница загружается
2	Ввести сумму в поле торговли	Сумма отображается
3	Нажать кнопку "Buy Yes"	Появляется модальное окно авторизации

Сценарий 5: Переход по несуществующей ссылке (негативный)

№	Действие	Ожидаемый результат
1	Ввести URL: polymarket.com/nonexistent	Открывается страница ошибки
2	Проверить отображение ошибки	Показывается 404 или сообщение об ошибке

4. Результаты автоматического тестирования

Скриншоты выполнения тестов:

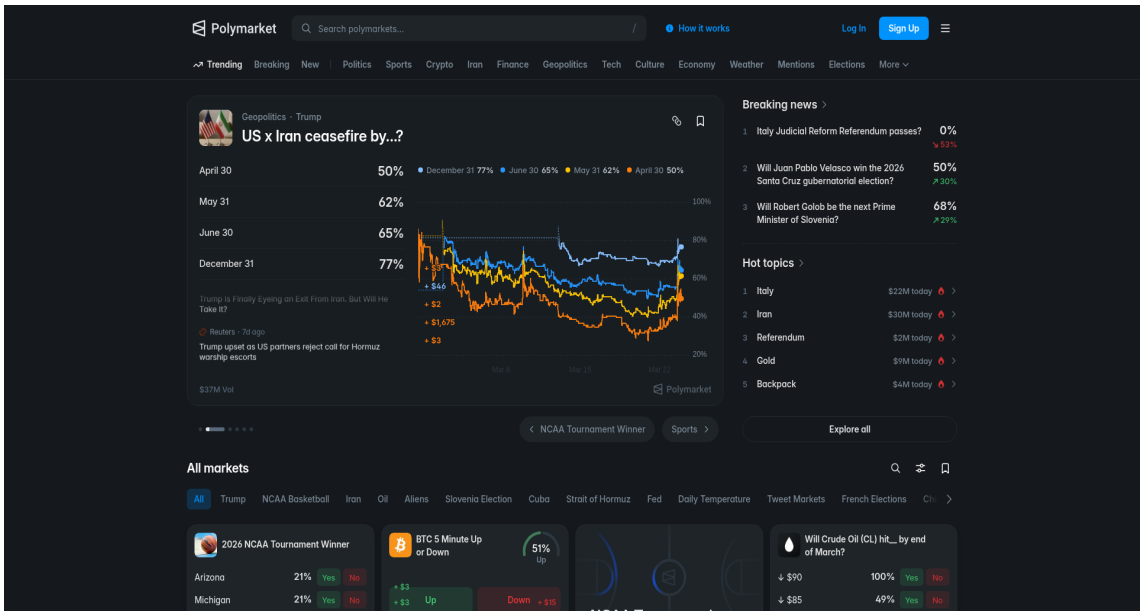

Рисунок 1 – 01 homepage

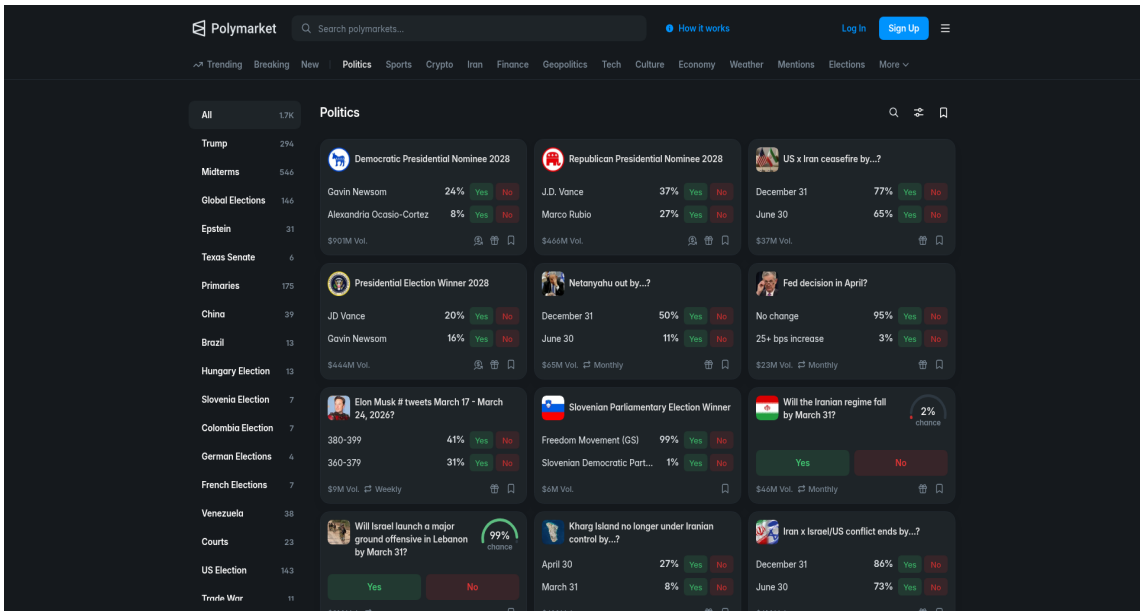

Рисунок 2 – 02 politics category

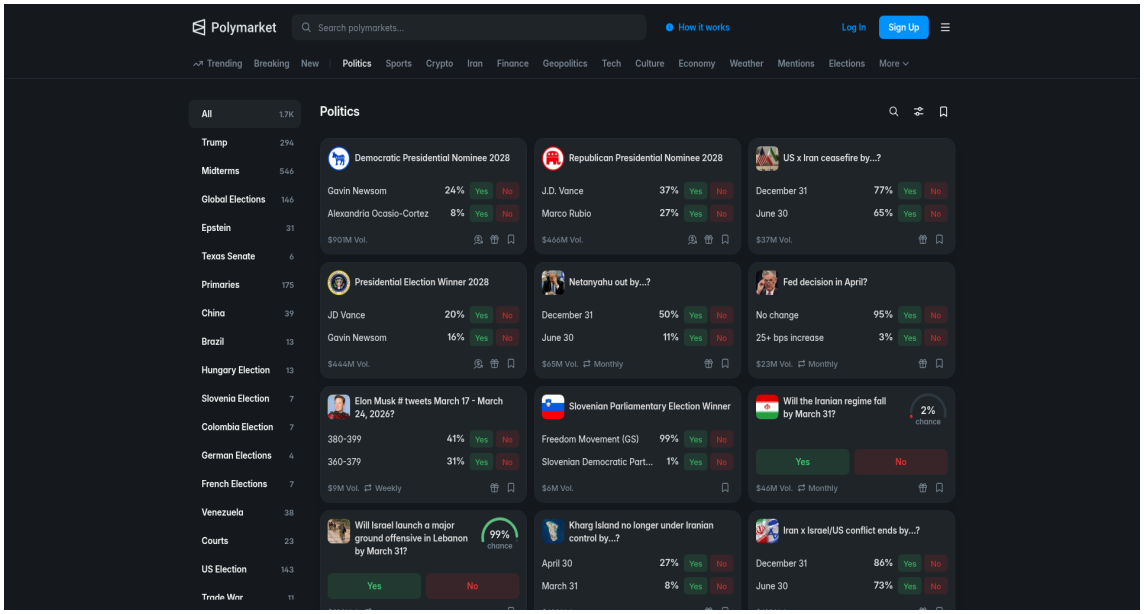

Рисунок 3 – 03 filtered results

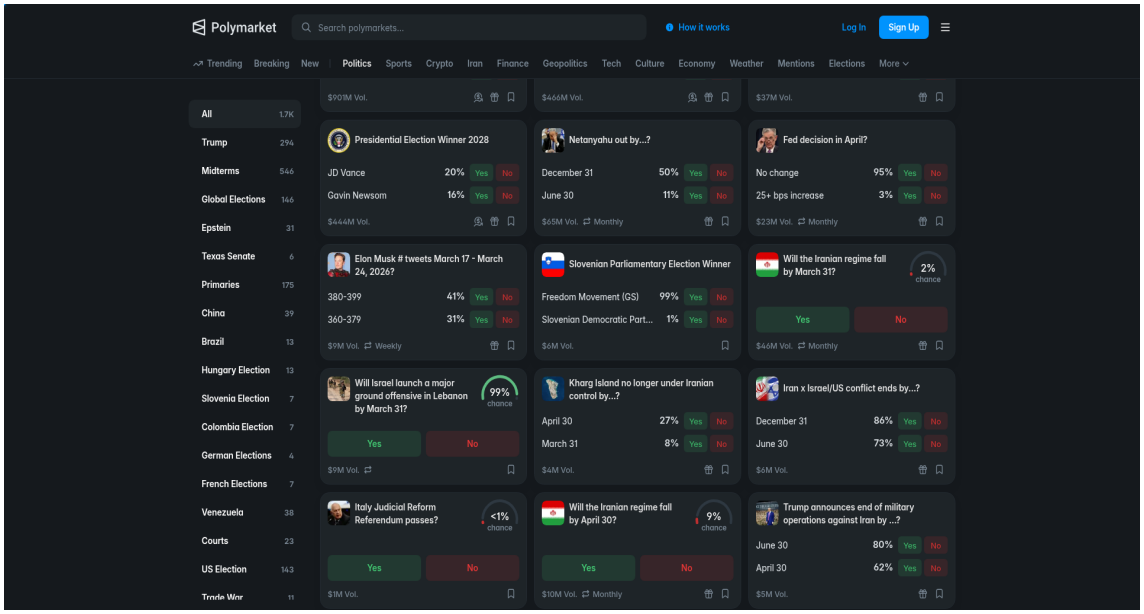

Рисунок 4 – 04 market page

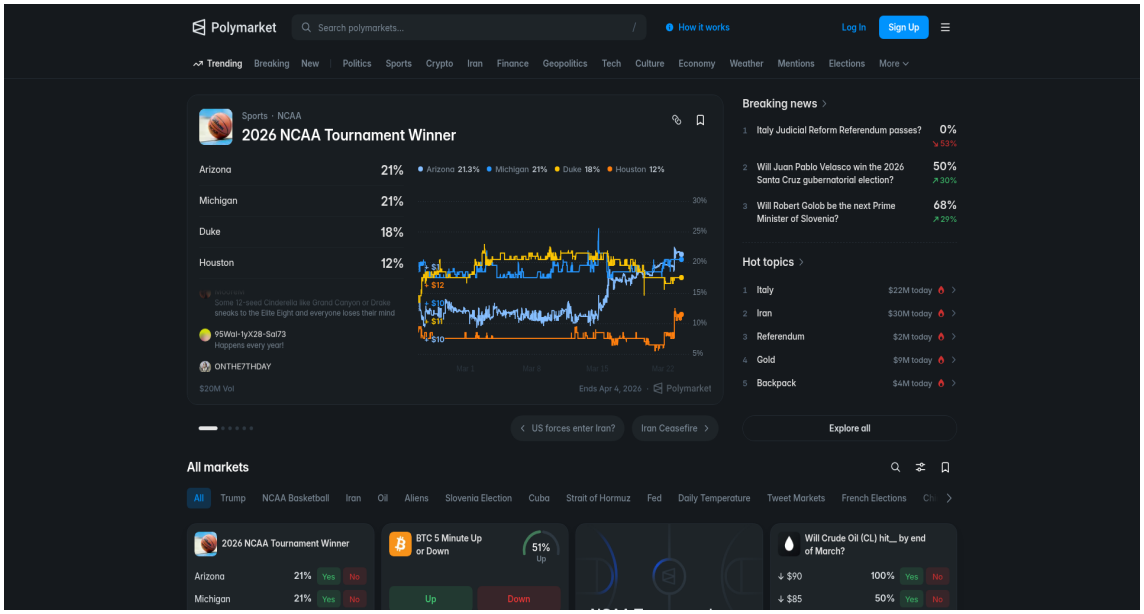

Рисунок 5 – 05 search home

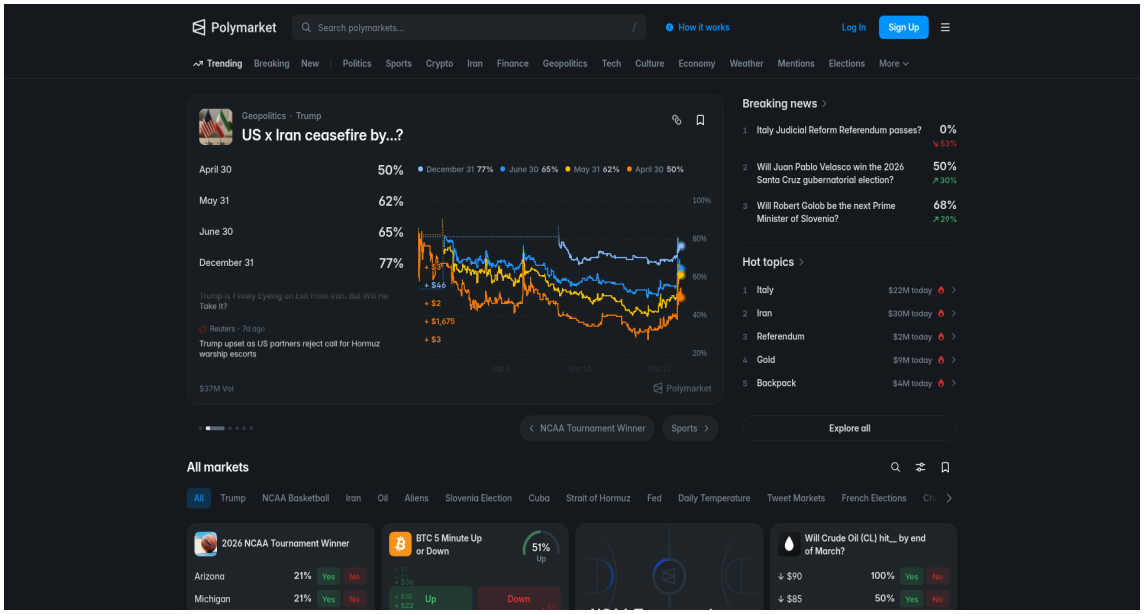

Рисунок 6 – 06 search clicked

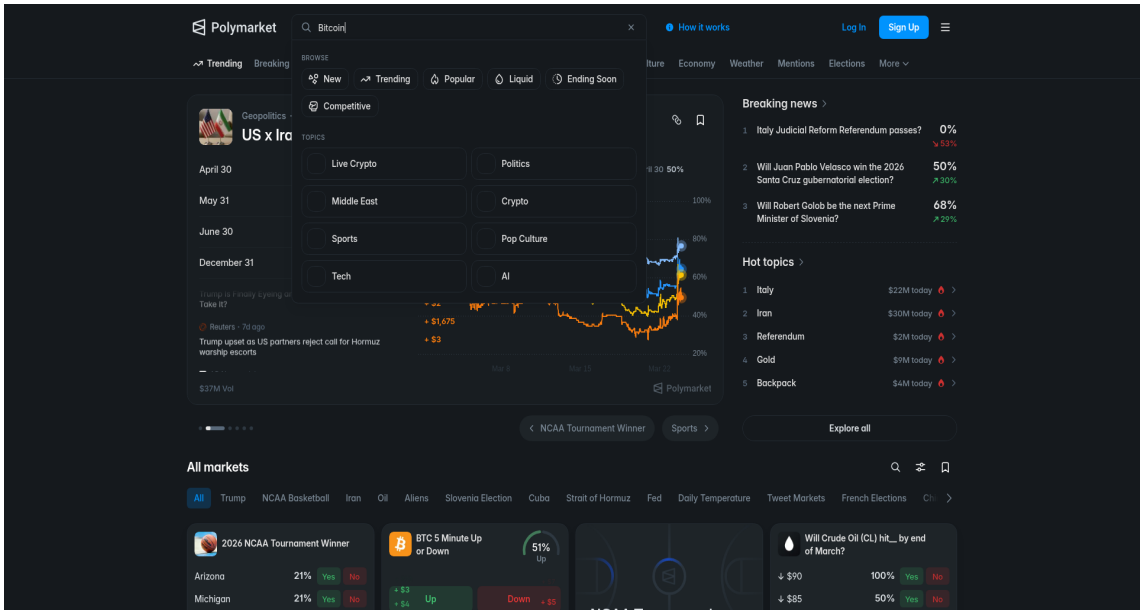

Рисунок 7 – 07 search typed

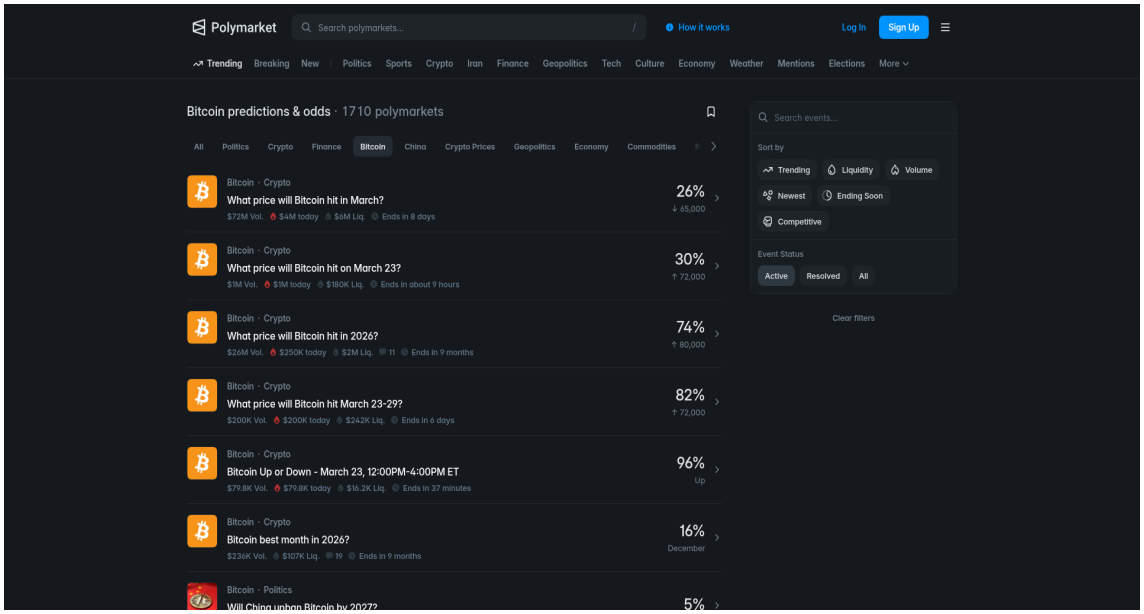

Рисунок 8 – 08 search results

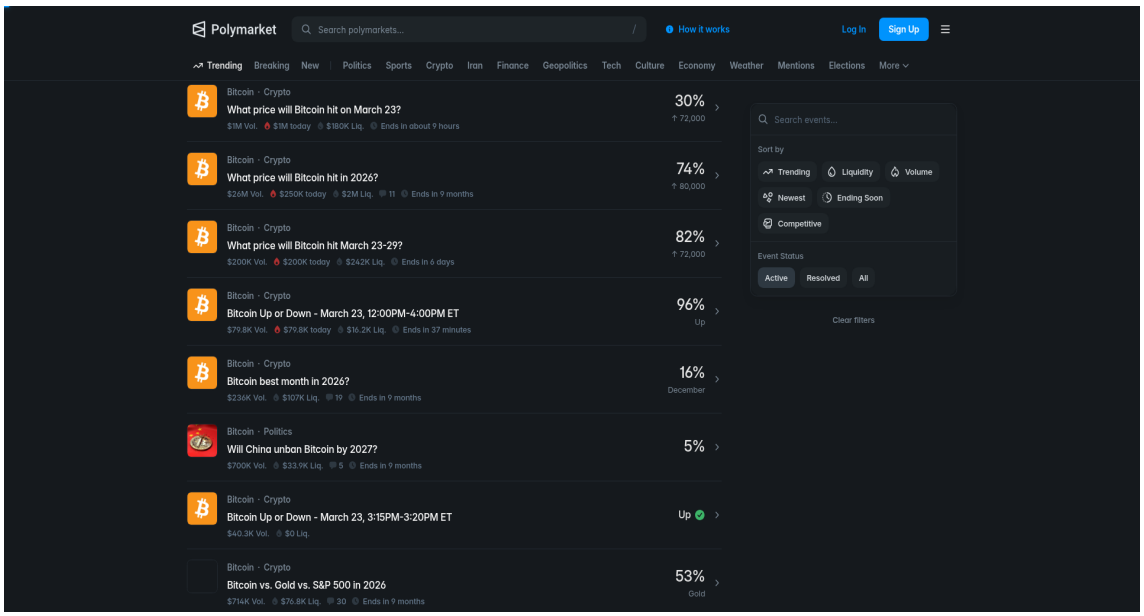

Рисунок 9 – 09 market opened

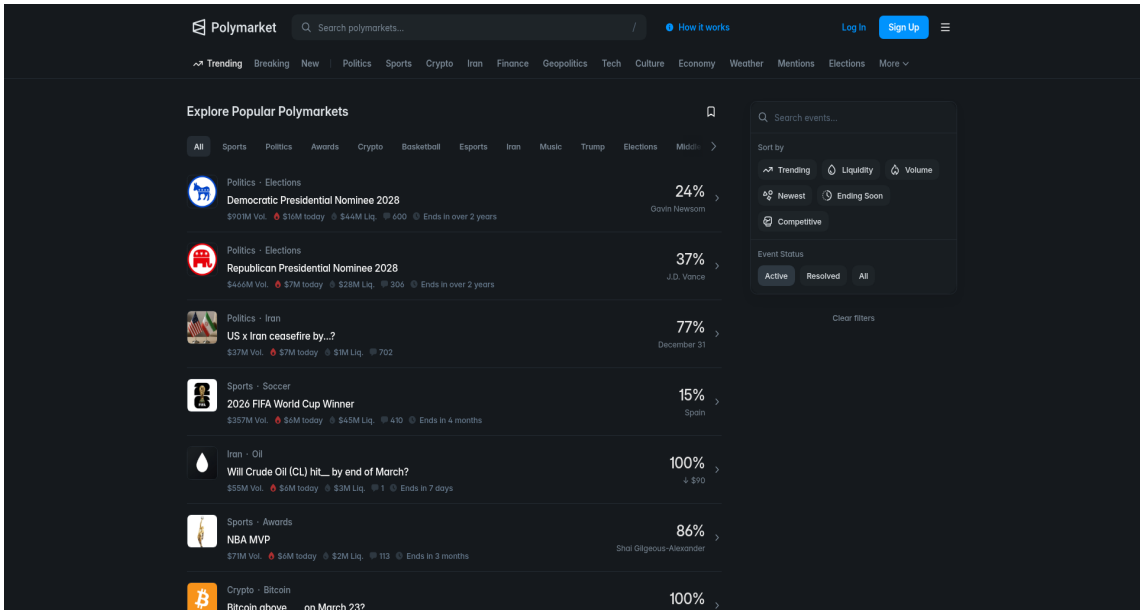

Рисунок 10 – 10 details home

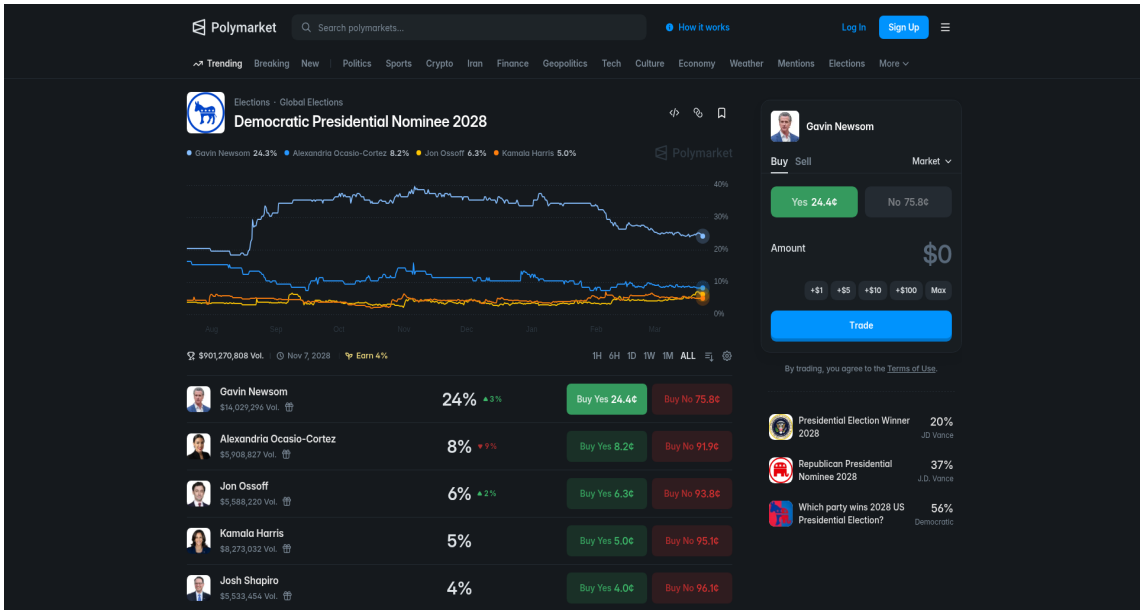

Рисунок 11 – 11 market details

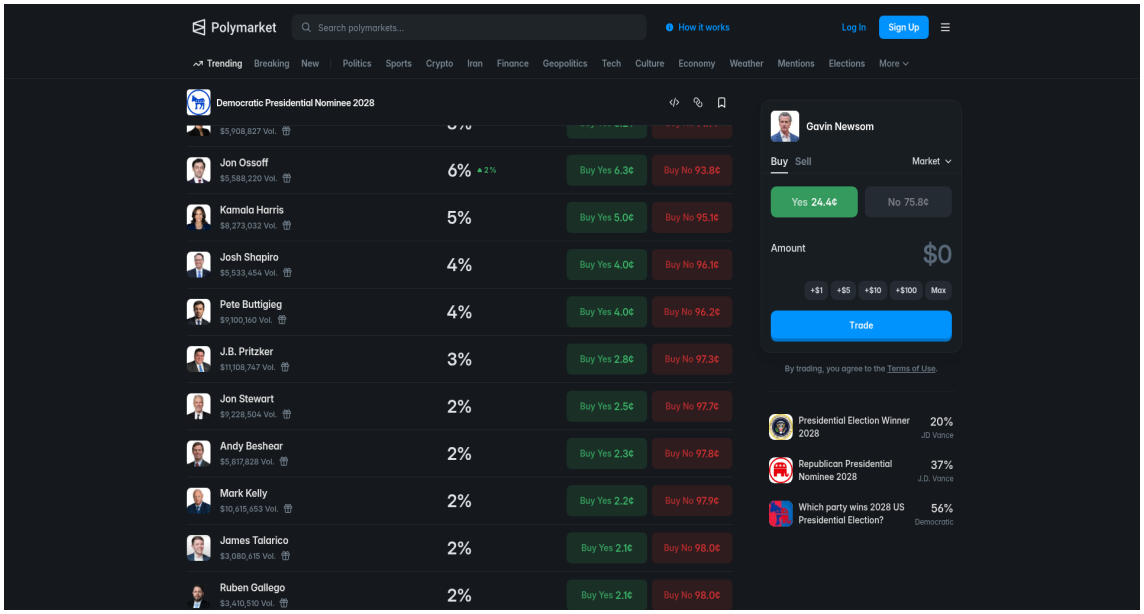

Рисунок 12 – 12 chart visible

Продолжение скриншотов:

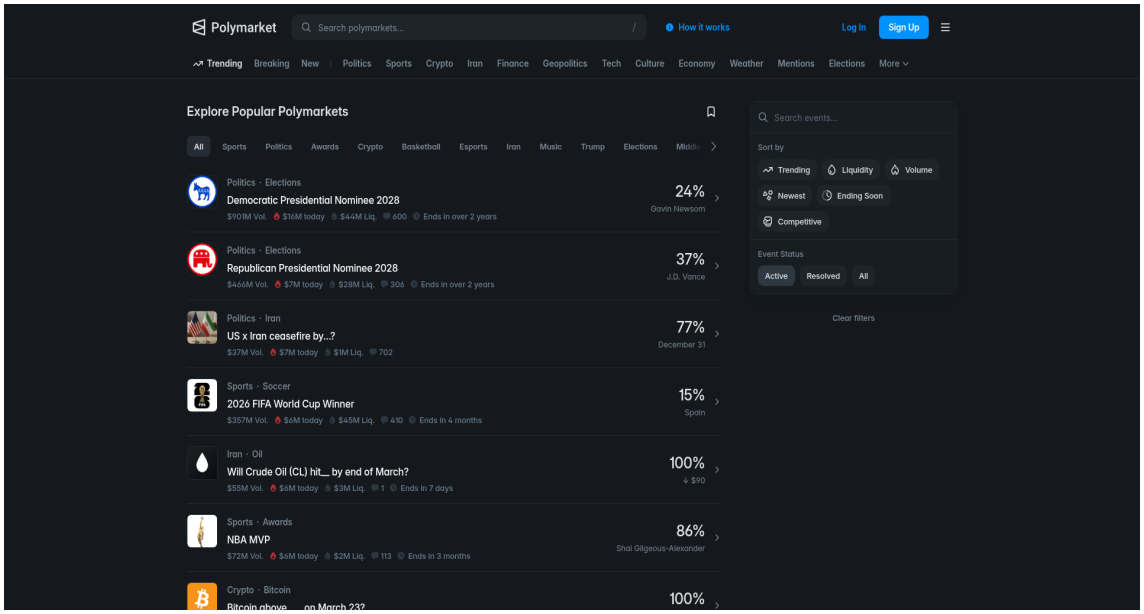

Рисунок 13 – 14 trade home

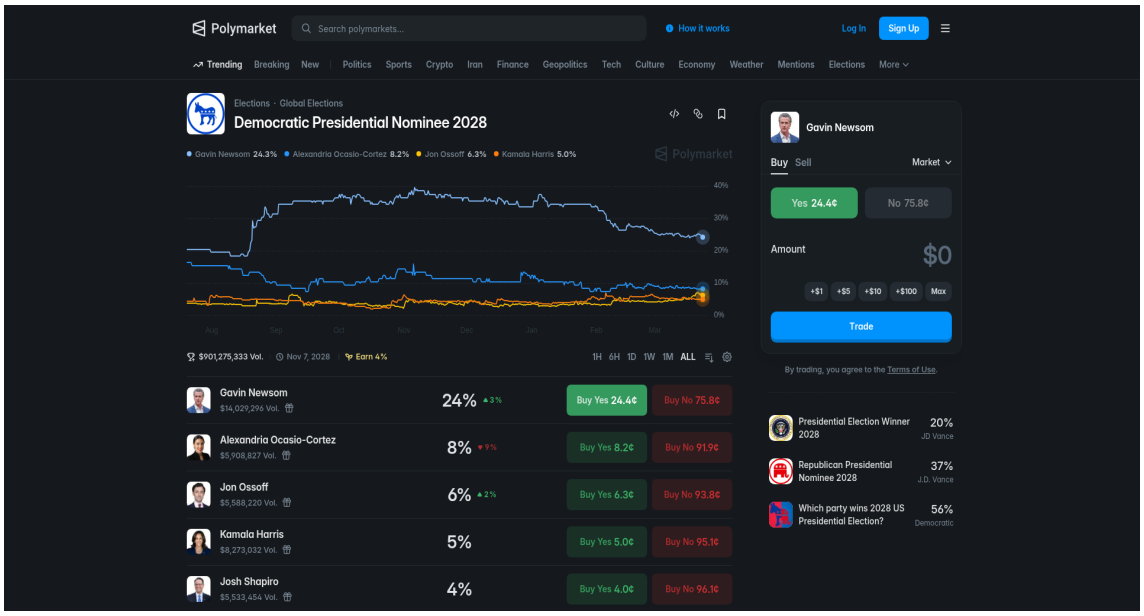

Рисунок 14 – 15 trade market

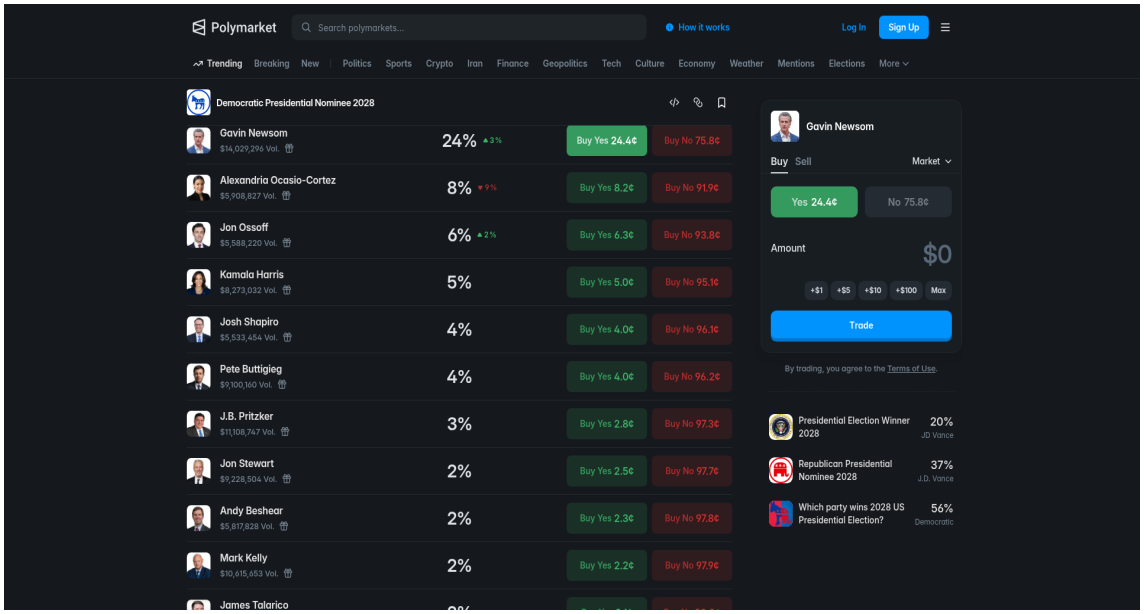

Рисунок 15 – 16 trade scrolled

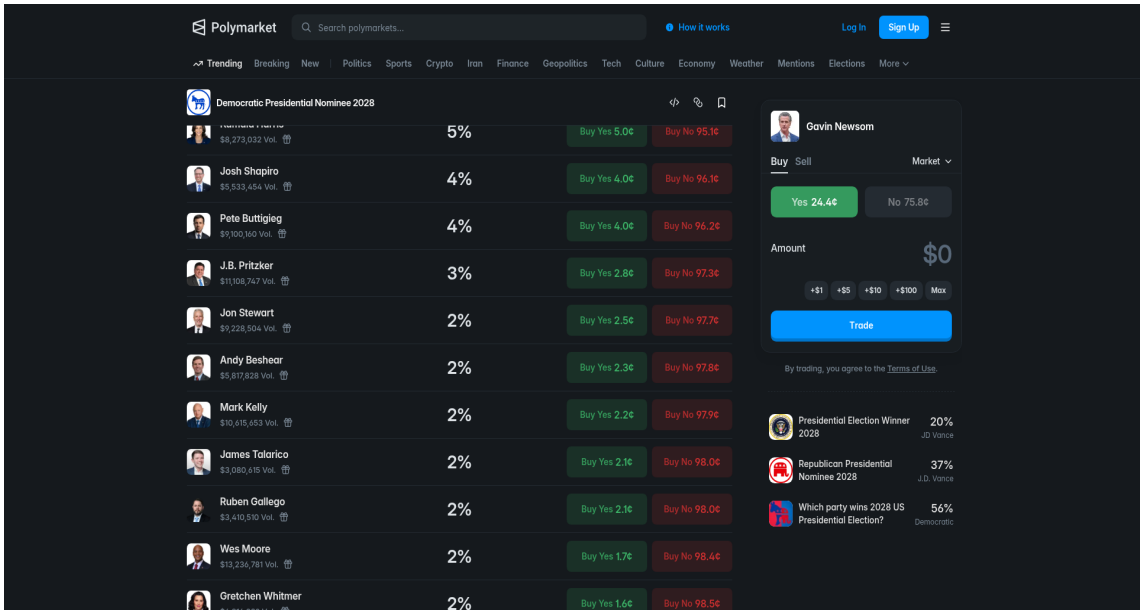

Рисунок 16 – 17 auth modal

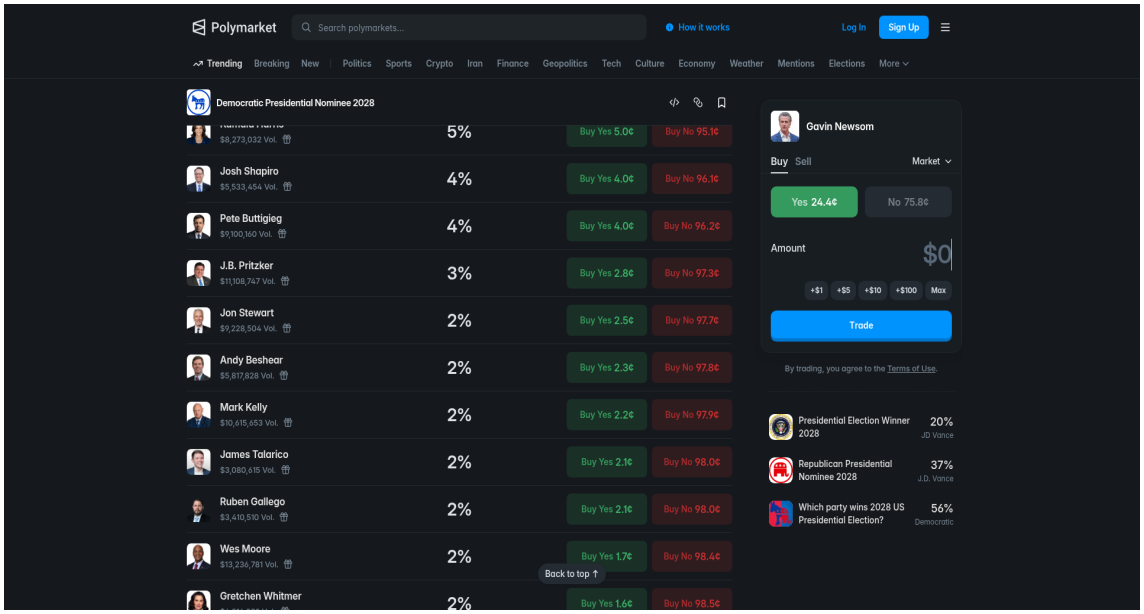

Рисунок 17 – 17 trade blocked

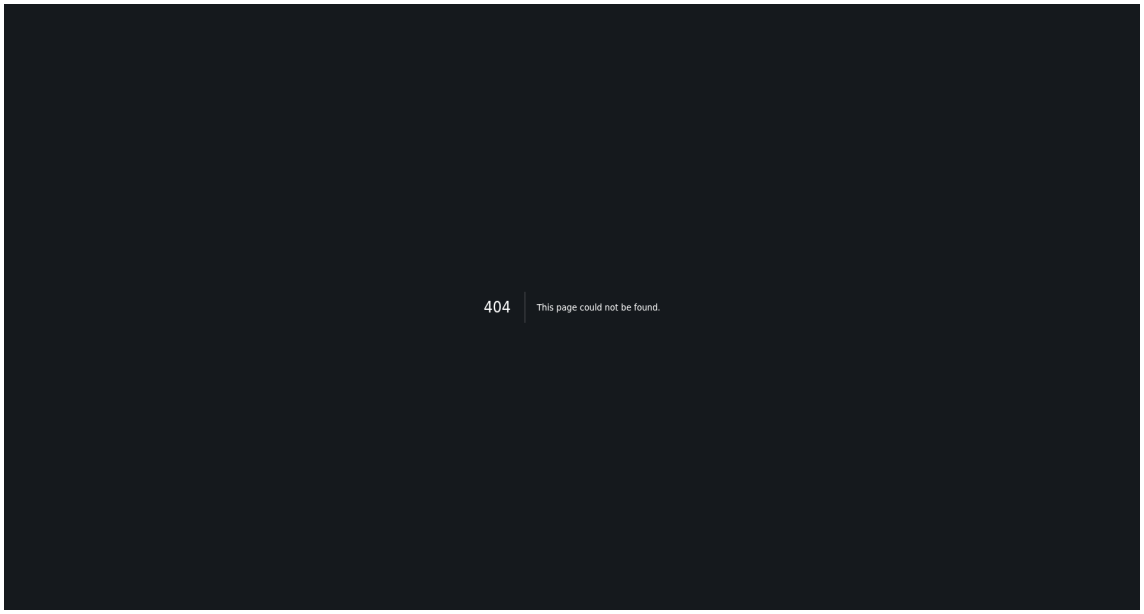

Рисунок 18 – 18 404 page

5. Листинг кода тестов

Ниже представлен исходный код автоматизированных тестов на Python с использованием Selenium:

Настройка браузера (conftest.py)

```
import pytest import os from selenium import webdriver from selenium.webdriver.chrome.service import Service from selenium.webdriver.chrome.options import Options @pytest.fixture(scope="session") def browser(): options = Options() options.binary_location = "bin/chrome-linux64/chrome" options.add_argument("--headless=new") options.add_argument("--no-sandbox") options.add_argument("--disable-dev-shm-usage") options.add_argument("--window-size=1920,1080") service = Service(executable_path="bin/chromedriver-linux64/chromedriver") driver = webdriver.Chrome(service=service, options=options) yield driver driver.quit()
```

Пример теста поиска

```
def test_search_market(self, browser): driver = browser driver.get("https://polymarket.com") wait = WebDriverWait(driver, 10) search_input = wait.until(EC.element_to_be_clickable( (By.CSS_SELECTOR, "input[type='search']") )) search_input.send_keys("Bitcoin") search_input.send_keys("\n") driver.save_screenshot("output/search_results.png")
```

Пример негативного теста

```
def test_trade_without_auth(self, browser): driver = browser driver.get("https://polymarket.com/markets") buy_btn = driver.find_element(By.XPATH, "//button[contains(text(), 'Buy')]") driver.execute_script("arguments[0].click();", buy_btn) modal = WebDriverWait(driver, 5).until( EC.presence_of_element_located((By.CSS_SELECTOR, "[class*='modal']")) ) assert modal is not None
```

6. Логи выполнения

```
============================= test session starts ============================== platform linux -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 cachedir: .pytest_cache rootdir: /dev/shm/polymarket collected 5 items tests/test_polymarket.py::TestCategoryFilter::test_view_markets_by_category PASSED [ 20%] tests/test_polymarket.py::TestSearch::test_search_market PASSED [ 40%] tests/test_polymarket.py::TestMarketDetails::test_view_market_details PASSED [ 60%] tests/test_polymarket.py::TestTradeWithoutAuth::test_trade_without_authorization PASSED [ 80%] tests/test_polymarket.py::Test404Error::test_nonexistent_page PASSED [100%] ========================== 5 passed in 177.19s (0:02:57) ===========================
```

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были автоматизированы пять тестовых сценариев для веб-платформы Polymarket с помощью инструмента Selenium и фреймворка Pytest. Тесты успешно выполнены в автоматическом режиме.

Все позитивные сценарии (фильтрация по категории, поиск, просмотр деталей рынка) выполнены успешно. Негативные сценарии (попытка торговли без авторизации, переход по несуществующей ссылке) также отработали корректно – система блокирует действия неавторизованных пользователей и корректно обрабатывает ошибки 404.

Selenium показал себя как надёжный инструмент для автоматизации тестирования веб-приложений. Комбинация Selenium + Pytest + локальные Chrome/ChromeDriver обеспечивает стабильное выполнение тестов.

Показатель	Значение
Всего тестов	5
Пройдено	5
Провалено	0
Процент успеха	100%
Время выполнения	177 сек